MEGATHON X: SINGULARITY – MILESTONE 1: APPROACH

Track: Accessible AI for All | **Team ID:** 320

Abstract: Dual-Track Multilingual Accessibility Architecture
Standard AI assistants assume users can see, hear, type, and read English. To target the highest-weighted criteria—output accessibility (25%), condition depth (20%), and Bhashini quality (20%)—Team 320 formulates two complementary solutions across Indic languages (English, Hindi, Telugu):

- **Track 1 – SunoShop (Use Case 2: eCommerce):** A voice-first PWA decoupling raw DOM structures from auditory browsing via aspect-based review distillation, Entity Masking for MT, and an offline hybrid RAG router.
- **Track 2 – Oriel (Use Case 3: City Signage):** A zero-tap, verify-then-guide multimodal sign reader using multi-frame agreement, rule-based aiming coaching, and Unicode locking to speak only confirmed transit facts.

Phase 1 testing (10:00 PM – 2:00 AM) benchmarks both solutions to converge on the optimal single track for Milestone 2.

1 1. Problem Statement & Domain Understanding

Conventional interfaces fail persons with disabilities (PwDs) and elderly citizens in two critical ways:

- **Input Gap:** Blind and motor-impaired users cannot aim cameras or tap small UI elements. 51.6% of photos taken by blind users suffer quality flaws, mostly framing and blur (VizWiz, CVPR 2020). Drag-and-drop checkout flows create severe motor friction (WCAG 2.5.7).
- **Output Gap:** Raw DOM trees and vision models output dense text, unlabelled tables, and code-mixed hallucinations. Transliteration often corrupts transit route numbers and brand names into phonetic gibberish (NetraLink, ICVGIP 2025).
- **Target Profiles: Visual (Mandatory):** Screen-reader parity, 3D spatial audio, auditory cues; **Ageing:** Slower chunked speech (0.75×), large high-contrast text, "Explain Simpler"; **Hand-Motor:** Zero-tap auto-capture, large single-switch targets, no drag-and-drop; **Hearing (Bonus):** Synchronized captions and haptic pulses.
- **Condition Inference:** Inferred via explicit commands and interaction friction (e.g., repeated ASR timeouts, tap misses, or clarification requests) to transparently offer UI adaptations without silent, unconsented context switching.

2 2. Track 1: SunoShop – Voice-First eCommerce Assistant (Use Case 2)

2.1 Core Concept: SunoShop is a multilingual, voice-first PWA coexisting with native screen readers (TalkBack, NVDA). It replaces linear DOM reading with structured auditory digests and isolates transactions within a hallucination-free state machine.

2.2 Architectural Pipeline:

- **Catalog Ingest & Offline Caching:** Product specs and prices are normalized in IndexedDB for instant lookup. Reviews are chunked by aspect and embedded for cosine-similarity retrieval, preventing the LLM from hallucinating untrusted claims.
- **Entity Masking Layer:** Bhashini ASR captures speech (≤ 20 s). To prevent MT from corrupting brand names or prices, an entity mask replaces proper nouns and numbers with tokens before translation, restoring them before Bhashini TTS.
- **Deterministic Rule Router:** Intercepts simple commands (*repeat, slower, simpler, track order*) to bypass the LLM completely, minimizing latency. Complex reasoning (e.g., comparing two items) invokes the LLM strictly over retrieved context.
- **Auditory Review & Sizing Engine:** Synthesizes reviews into plain-language Pros, Cons, and Accessibility Verdicts. Product images and size charts are pre-processed offline via VLMs and OCR for instant auditory recommendations.
- **Order Management State Machine:** Checkout, tracking, and returns run on a deterministic state machine. Checkout requires a dynamic spoken confirmation phrase in the user's native language.

2.3 Condition Adaptations: Blind-first semantic headings, live regions, 3D spatial audio (Web Audio API panning search results left-to-right); Ageing mode with reduced speech cadence and high-contrast text; Hand-motor voice checkout eliminating multi-step cart funnels.

3 3. Track 2: Oriel – Zero-Tap City-Sign Reader (Use Case 3)

3.1 Core Concept: Oriel reads street and transit signs with zero taps. Its core is *verify-then-guide*: it speaks only facts confirmed across multiple sharp frames (route, destination, platform, arrow), marks unverified text as uncertain, and actively coaches the user when a key fact is unclear (*"I can read Secunderabad but not the route number. Hold still, zooming in"*). It reads signs; it never plans routes.

3.2 End-to-End Architectural Pipeline:

1. **Watch (On-Device):** Streams low-resolution frames, tilt/motion sensors, and optional head tracking (MediaPipe Face Landmarker).
2. **Trigger:** Auto-capture on sharp, steady, text-like frame; voice "read"; head dwell; or a single massive button.
3. **Quality Gate & Active Guidance (Rules, No LLM):** Detects tilt (ground/sky), blur, darkness, or glare. Issues spoken guidance via pre-recorded Bhashini clips in 5-second sweeps (*"Slowly turn left, I'll say stop"*); caps at 3 attempts before honest abstention.
4. **Burst & Crop:** Captures a 1-second burst at high resolution, retains the 3 sharpest frames, and crops around text to eliminate tremor.
5. **Read Engine:** Gemini Flash-Lite extracts structured JSON per frame (type, original script, route, destination, arrows); secondary verification via PaddleOCR and Bhashini OCR.
6. **Multi-Frame Verification:** A field is confirmed only when 2 of 3 frames agree and Unicode script is valid; otherwise re-reads or abstains.
7. **Render & Locked Speech Synthesis:** Chained Bhashini Translation+TTS locks route numbers and proper nouns against translation corruption; provides synchronized on-screen captions and a two-way speech relay ("Ask someone nearby").

3.3 Condition Adaptations: Visual: key fact first, "more"/"spell it", confirmed facts vs. "it looks like"; Ageing: help ladder, long timeouts, chunked speech with pauses; Hand-Motor: zero-tap auto-capture, burst against tremor, large dwell targets.

4 4. Graceful Degradation, Offline Fallbacks & Tech Stack

- **ASR Misrecognition:** Low transcript plausibility triggers a polite "Check Again" protocol to confirm intent before taking action.
- **Offline Fallback Ladder:** Both solutions cache workflows in IndexedDB via Service Workers. If Bhashini times out or network drops, the system falls back instantly to on-device browser `speechSynthesis` and Tesseract.js OCR, displaying an offline banner.
- **Tech Stack:** React + Vite PWA (Android/Laptop), FastAPI, Gemini Flash-Lite, Bhashini APIs (ASR, NMT, TTS), MediaPipe Tasks Vision, Web Audio API, HTML5 Vibration API, IndexedDB.

5 5. Planned Testing Methodology & Milestone Execution Plan

5.1 Quantitative Benchmarks & Testing Strategy:

- **Safety & Accuracy Metrics:** Key-fact accuracy, detection rate, correct abstention rate, and **wrong-but-confident rate** (headline safety metric).
- **Usability & Performance:** Side-by-side comparison of raw DOM vs. SunoShop (Time-to-Information, words listened); zero-tap auto-capture success rate; end-to-end Bhashini roundtrip latency ($< 1.5\,s$).
- **Datasets & Assistive Tech Trials:** 30–40 real Hyderabad transit/street signs (Telugu, Hindi, English) with labeled ground truth; 50 multi-attribute eCommerce products; screen reader audits with TalkBack and NVDA; 3–5 user trials (including blindfolded teammates).

5.2 Execution Roadmap (M1 → M2 → Final):

Milestone	Key Action Items for Dual Solutions	Deadline
Phase 1 (10PM–2AM)	Connect Bhashini ASR/NMT/TTS endpoints; implement SunoShop Entity Masking & Oriel multi-frame verify logic	11 Oct, 02:00 AM
M2 Milestone	Run comparative benchmarks on sign dataset and eCommerce flows; select primary track or hybrid; code push	11 Oct, 04:00 AM
Phase 2 (Final)	Refine chosen track, complete TalkBack audits, record 4-minute captioned demo video, finalize report	11 Oct, 11:00 AM